

TASK 2 – Identify the Basic Operation

1. Linear Search — Iterative

Basic operation: Comparison between the registration number and the target number.

Where it occurs:

The comparison occurs inside the for loop:

```
if (records[i] == key)
```

Why it is important:

This comparison is performed repeatedly while checking each student record. Therefore, it is the operation that mainly determines the execution time of linear search.

Effect of increasing n:

If the number of student records increases, the algorithm may need to check more records.

Best case → 1 comparison

Average case → approximately $n/2$ comparisons

Worst case → n comparisons

Therefore, the number of comparisons increases approximately linearly with n .

2. Binary Search — Recursive

Basic operation: Comparison between the middle registration number and the target number

Where it occurs:

The comparison occurs after calculating the middle element:

if (records[mid] == key)

and also:

if (key < records[mid])

Why it is important:

Each comparison eliminates approximately half of the remaining records. Therefore, this comparison determines how quickly the search space becomes smaller.

Effect of increasing n:

For every recursive call, the search range is reduced approximately by half.

For example:

$n = 16$

$16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$

So the number of comparisons grows approximately as:

$\log_2(n)$

Even when the database becomes very large, the number of comparisons increases slowly.

1. Hash Search

Basic operation: Comparison between the value stored in the hash table and the target registration number.

Where it occurs:

The comparison occurs inside the while loop:

```
if (hashTable[index] == key)
```

The hash function also performs the calculation:

```
index = key % SIZE;
```

Why it is important:

Hashing attempts to directly locate the position of the registration number instead of checking every record. The comparison determines whether the required record has been found.

If a collision occurs, the algorithm checks the next positions using:

```
index = (index + 1) % SIZE;
```

Effect of increasing n:

With a properly designed hash table and a suitable load factor, the average number of comparisons remains small even when the number of records increases.

However, if many records map to the same location, collisions increase and more comparisons are required.

Summary Table

Algorithm	Basic Operation	Where It Occurs	Effect as n Increases
Linear Search	Compare records[i] with key	Inside for loop	Comparisons increase linearly
Binary Search	Compare records[mid] with key	Each recursive call	Comparisons increase logarithmically
Hash Search	Compare hashTable[index] with key	Inside while loop	Usually remains nearly constant; increases with collisions

Final observation

For a large student database, the basic operation behaves very differently in the three approaches. **Linear Search** may perform up to n comparisons, **Binary Search** requires about $\log_2 n$ comparisons after sorting, while **Hash Search** can locate a record with approximately constant average search effort when the hash table is well designed. This difference becomes increasingly important as the database grows from 1,000 to 1,000,000+ records.